

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

INTRO TO CRITICAL INFRASTRUCTURE FOR IT LEARNERS

LAB 02 — PLC PROGRAMMING LANGUAGES

Name: Sidra Shaikh | Registration No: C11/26/ACIS/17349 | Date: 20 September 2026

Student Edition

Authorized ICDFA training environments only
icdfa.edu.ng

Lab 02: PLC Programming Languages

Course: Intro to Critical Infrastructure for IT Learners | **Estimated Duration:** 2–3 hours

Submission deadline: As published in ICDFA E-Campus.

Authorization & Safety: Perform all activities only on the ICDFA-provided virtual machines, packet captures and isolated training networks. Do not bridge these VMs to production networks, scan external systems, or reuse credentials observed in training data.

Learning Outcomes

- Compare the PLC programming languages presented in the course.
- Interpret a motor start/stop latching circuit in Ladder Diagram.
- Recognize equivalent motor-control logic represented using Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- Explain how tags and Boolean data types support PLC logic.

Laboratory Environment

System	Expected Address	Account	Password
PLC VM	192.168.1.10	plc	password

Note: If the instructor-provided VM image uses updated addressing or credentials, use the values published in E-Campus. Do not alter addressing unless instructed.

Pre-Lab Checklist

- Read the complete lab before beginning.
- Confirm you can access the required VM(s) and that virtualization support is enabled.
- Use the instructor-specified isolated/host-only network.
- Create or restore the clean starting snapshot for this lab.
- Prepare a report document with your name, registration number, course, lab number and date.
- Capture evidence as you progress rather than repeating the lab at the end.

Procedure

Step 1. Start from the clean ICDFA PLC VM snapshot and sign in to the PLC training environment.

Step 2. Launch OpenPLC Editor and open the Ladder Diagram example from the PLC Programming Language Examples directory.

Step 3. Open Res0 > instance0 (Motor_Control). Identify the normally open Start contact, normally closed Stop contact, MotorOutput latch contact and MotorOutput coil.

Step 4. Open the tag/variable list. Confirm that the motor-control tags use Boolean values and record the purpose of each tag in your worksheet.

Step 5. Return to File > Open and load the Function Block Diagram example. Open Motor_Control_FBD and identify the block that maintains the motor output until a stop condition occurs.

Step 6. Open the Structured Text example and inspect MotorControlST. Describe how the same start/stop behavior is represented using text rather than graphical contacts and coils.

Step 7. Open the Instruction List example and inspect MotorControl. Record the main visual difference between this low-level textual representation and Structured Text.

Step 8. Open the Sequential Function Chart example and inspect MotorControl_SFC. Identify how steps/transitions express process sequence.

Step 9. Complete the language comparison matrix in the lab report for all five representations.

Step 10. Capture one evidence screenshot for each programming representation, ensuring the program name is visible.

Required Evidence & Submission

- Five screenshots: Ladder Diagram, Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart.
- A completed comparison matrix covering representation, strengths, limitations and a suitable industrial use case.
- A short explanation showing how the same motor start/stop requirement can be represented differently while preserving the same control objective.

Recommended filename: <RegistrationNo>_LAB02_PLC_Programming_Languages.pdf

Solution:

1. Laboratory Environment

The ICDFA-provided PLC virtual machine was started from the clean lab snapshot, and OpenPLC Editor was launched inside the guest operating system. All five programs in this report were opened from the supplied PLC Programming Language Examples directory. No addressing or credentials were altered.

2. Evidence: Programming Representations of the Motor Start/Stop Logic

The same control requirement is examined in five IEC 61131-3 representations: the motor starts when Start is pressed, keeps running after Start is released, and stops when Stop is pressed. The programs opened in OpenPLC Editor were Motor_Control (Ladder Diagram), Motor_Control_FBD, Motor_Control_ST, Motor_Control_IL and Motor_Control_SFC.

2.1 Ladder Diagram (LD): Motor_Control

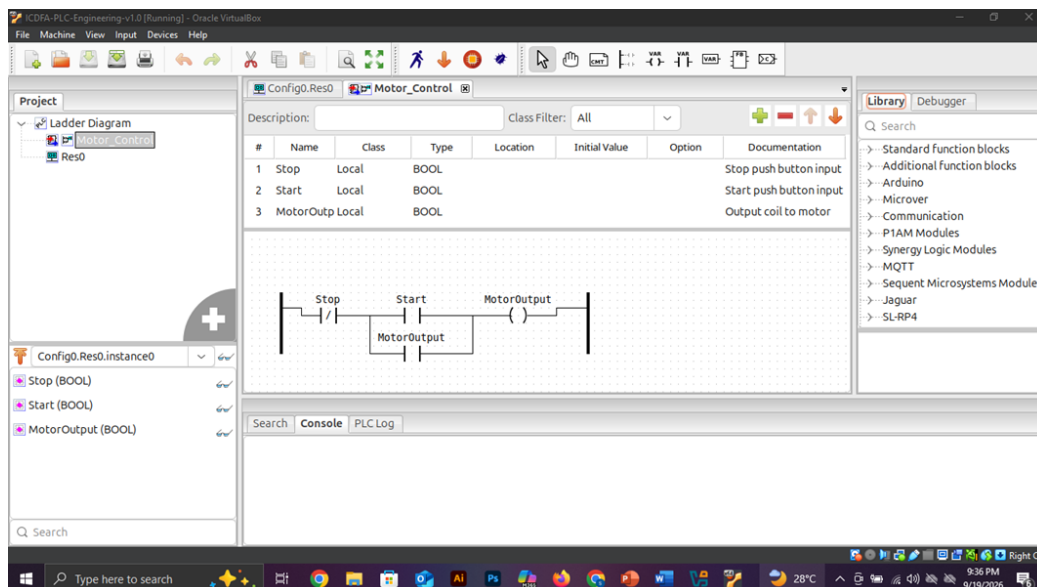


Figure 1: Ladder Diagram program Motor_Control in OpenPLC Editor.

The rung, read from the left power rail to the right power rail, contains four elements:

- **Stop:** a normally closed (NC) contact in series with the rest of the rung. It passes power until the Stop input goes TRUE.
- **Start:** a normally open (NO) contact that closes only while the Start button is pressed.

- **MotorOutput (contact):** a NO contact in parallel with Start. It is the latch (seal-in) that keeps the rung energized after Start is released.
- **MotorOutput (coil):** the output that drives the motor.

2.2 Tag List (Boolean Variables)

The tag list of the Motor_Control program contains three Local variables, all of type BOOL:

Tag	Type	Purpose
Start	BOOL	Start push button input. TRUE while the button is pressed.
Stop	BOOL	Stop push button input. TRUE when the stop condition is active.
MotorOutput	BOOL	Output coil to the motor. Also used as the latch (feedback) condition in the logic.

Boolean values are used because each signal has only two states (ON/OFF, TRUE/FALSE), which is exactly what a discrete start/stop function requires. The same three BOOL tags appear in all five programs. In the FBD, MotorOutput is documented as the latched motor output, and in the SFC, Start and Stop are transition inputs and MotorOutput is TRUE while the Running step is active.

2.3 Function Block Diagram (FBD): Motor_Control_FBD

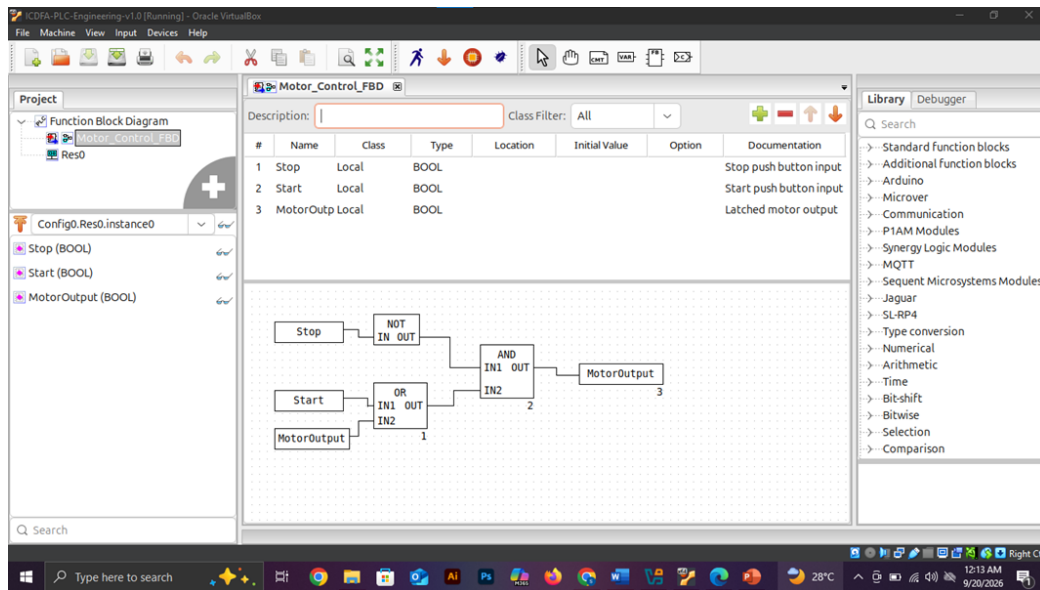


Figure 2: Function Block Diagram program Motor_Control_FBD in OpenPLC Editor.

In Motor_Control_FBD the logic is drawn as three connected function blocks instead of contacts and coils. The Stop input passes through a NOT block. The Start input and the MotorOutput variable both feed an OR block (execution order 1). The output of the NOT block and the output of the OR block feed an AND block (order 2), and the result is assigned to MotorOutput (order 3).

The block that maintains the motor output until a stop condition occurs is the OR block. Because MotorOutput is fed back into its second input, the output stays TRUE after Start is released, which is the same seal-in action as the

parallel MotorOutput contact in the ladder. The AND block with the inverted Stop signal forces MotorOutput to FALSE as soon as Stop becomes TRUE.

2.4 Structured Text (ST): Motor_Control_ST

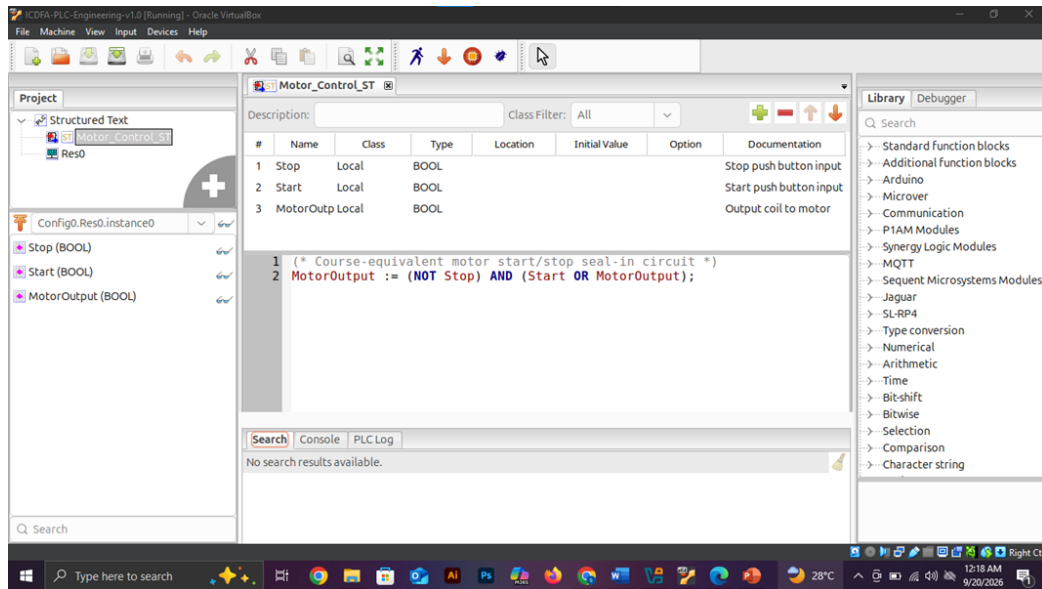


Figure 3: Structured Text program Motor_Control_ST in OpenPLC Editor.

In Structured Text the same behaviour is written as a text statement rather than drawn with graphical contacts and coils. A single Boolean assignment ($:=$) sets the output: MotorOutput becomes TRUE only when Stop is not TRUE and either Start is TRUE or MotorOutput is already TRUE. The MotorOutput term on the right-hand side is the latch. The program contains:

```
(* Course-equivalent motor start/stop seal-in circuit *)
MotorOutput := (NOT Stop) AND (Start OR MotorOutput);
```

2.5 Instruction List (IL): Motor_Control_IL

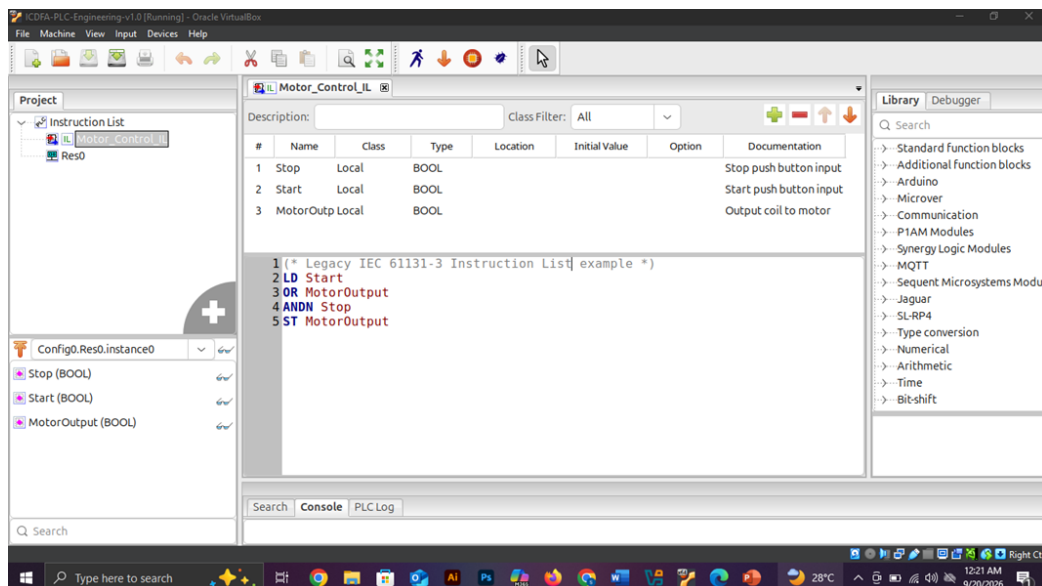


Figure 4: Instruction List program Motor_Control_IL in OpenPLC Editor.

Instruction List is a low-level textual representation. Each line holds a single operator and operand that acts on an accumulator. The program contains:

```
(* Legacy IEC 61131-3 Instruction List example *)
LD    Start
OR    MotorOutput
ANDN  Stop
ST    MotorOutput
```

LD loads Start into the accumulator, OR combines it with MotorOutput (the latch), ANDN combines the result with the inverse of Stop, and ST stores the final value in MotorOutput. This gives the same logic as the ST statement.

The main visual difference from Structured Text is that IL is a vertical list of short, one-operation instructions close to machine level, whereas ST reads like a high-level program with expressions, assignment (:=) and keywords such as IF and THEN.

2.6 Sequential Function Chart (SFC): Motor_Control_SFC

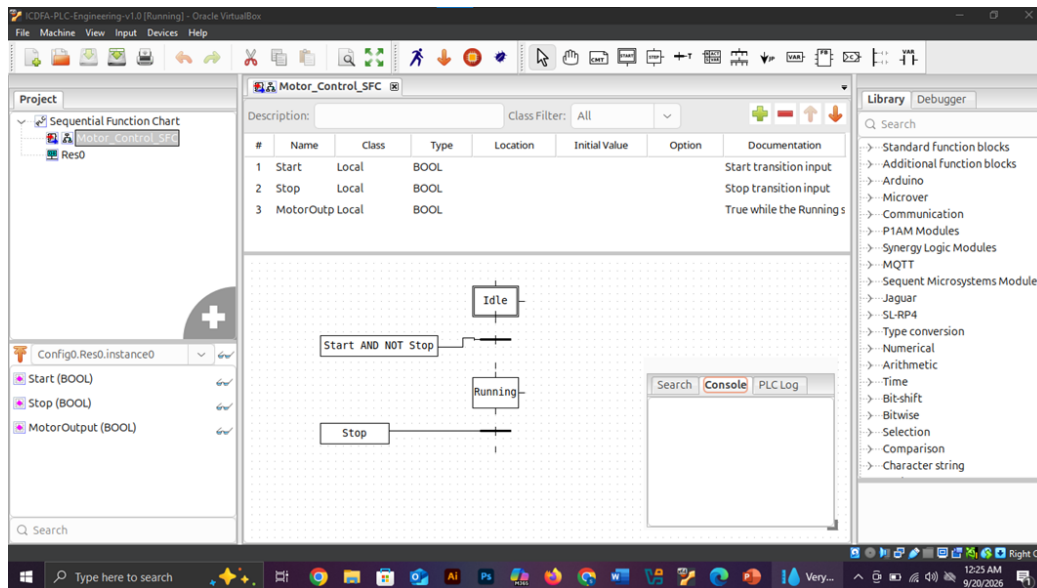


Figure 5: Sequential Function Chart program Motor_Control_SFC in OpenPLC Editor.

The SFC organizes the process as steps (states) connected by transitions (conditions). Motor_Control_SFC has two steps: Idle, the initial step drawn with a double border, and Running. The transition from Idle to Running has the condition Start AND NOT Stop, so the chart moves to Running only when Start is TRUE and Stop is not. The transition on the Stop condition ends the Running step and returns the sequence to Idle. MotorOutput is documented as TRUE while the Running step is active, so the motor runs for exactly as long as the chart is in that state.

Here the latching effect comes from the step itself staying active until its exit transition fires, rather than from feedback of the output. The order of the process is visible directly from the layout of the chart.

3. Language Comparison Matrix

Language	Representation	Strengths	Limitations	Suitable industrial use case
----------	----------------	-----------	-------------	------------------------------

Language	Representation	Strengths	Limitations	Suitable industrial use case
Ladder Diagram (LD)	Graphical contacts and coils between two power rails, modelled on relay wiring.	Familiar to electricians and technicians; easy to read and troubleshoot online.	Becomes large and hard to follow for calculations or complex logic.	Motor start/stop, interlocks and general machine control.
Function Block Diagram (FBD)	Graphical blocks (AND, OR, timers, latches) connected by signal lines.	Clear signal flow; reusable blocks; good for continuous signal processing.	Large diagrams become cluttered; harder to follow sequences.	Process control, PID loops and analog signal handling.
Structured Text (ST)	High-level text with assignments, IF/THEN, loops and Boolean expressions.	Compact; ideal for calculations, loops and complex conditions.	Steeper learning curve; less intuitive for technicians used to relay logic.	Data handling, recipe management and mathematical routines.
Instruction List (IL)	Low-level, assembly-style text with one instruction per line.	Very compact and close to how the PLC executes logic.	Hard to read and maintain; largely deprecated in modern standards.	Legacy systems and very small, simple programs.
Sequential Function Chart (SFC)	Graphical steps and transitions showing the order of a process.	Makes sequences and states explicit; easy to follow and debug.	Not suited to simple logic; step actions still need another language.	Batch processes, machine sequences, start-up and shut-down control.

4. One Requirement, Several Representations

All five programs implement the same control objective: the motor starts when Start is pressed, continues to run after Start is released, and stops when Stop is pressed. The languages differ only in how that objective is expressed.

In Ladder Diagram, a normally closed Stop contact is in series with a Start contact, which has a MotorOutput contact in parallel to latch it, and the combination drives the MotorOutput coil. The Function Block Diagram shows the same relationship as a NOT block on Stop, an OR block combining Start with MotorOutput feedback, and an AND block joining the two. Structured Text writes it as one Boolean assignment, `MotorOutput := (NOT Stop) AND (Start OR MotorOutput)`, and Instruction List as the sequence `LD Start, OR MotorOutput, ANDN Stop, ST MotorOutput`. The Sequential Function Chart describes it as two states, Idle and Running, joined by a Start AND NOT Stop transition and a Stop transition.

In every case the tags Start, Stop and MotorOutput keep the same meaning and the same Boolean type, so the resulting behaviour of the motor is identical regardless of the language chosen. The choice of language therefore depends on readability, the complexity of the logic and the skills of the people who will maintain it, not on the control objective itself.

5. Knowledge Check

1. Why is Ladder Diagram intuitive for technicians familiar with relay logic?

Response: Ladder Diagram looks like a relay wiring schematic, with contacts, coils and power rails. Technicians can read it and trace how power would flow through the circuit without having to learn a programming language, and they can see which contacts are open or closed when troubleshooting.

2. What is one advantage of Structured Text for complex calculations or conditions?

Response: Structured Text is compact and supports mathematical operations, comparisons, loops and IF/THEN structures. A calculation or a multi-part condition that would need many contacts or blocks in a graphical language can be written in a few lines, which is easier to read and change.

3. When might a Sequential Function Chart be more appropriate than a single ladder rung?

Response: A Sequential Function Chart is better when a process runs in ordered stages, such as filling, heating, mixing and draining. Each step and the condition for moving to the next is shown explicitly, whereas a single rung only expresses one logical condition and cannot show the sequence.

4. Why must the meaning of tags remain consistent when the representation/language changes?

Response: Tags are the shared names for the real inputs, outputs and internal states. If Start, Stop or MotorOutput meant something different in another language, the same program would behave inconsistently and could drive the motor incorrectly or unsafely. Keeping tag meanings and types constant means the control behaviour stays the same whichever language is used.

6. Conclusion

This lab showed that a single motor start/stop requirement can be written in Ladder Diagram, Function Block Diagram, Structured Text, Instruction List and Sequential Function Chart while keeping the same behaviour. Each language has its own strengths, and selecting the right one is a balance between readability, complexity and maintainability, while the tags and the control objective remain consistent.

Marking Rubric — 100 Marks

Criterion	Marks	Standard
Technical execution	50	Lab procedure completed correctly; required configuration/analysis works as intended.
Evidence	20	Screenshots/files are complete, readable, relevant and clearly labelled.
Analysis & knowledge check	20	Responses demonstrate understanding of the industrial-control concepts observed.
Professional reporting	10	Clear structure, correct terminology, concise writing, file naming and submission compliance.

Submission Integrity: Evidence must come from the student's own assigned laboratory environment. Screenshots should show enough context to verify the activity. Do not submit another learner's screenshots or project files.